# PACE: Exact Explanations, Trust Signals and Auditable Repair for Centroid-Based Fuzzy Min-Max Classifiers

## Abstract

Fuzzy min-max neural networks classify data with hyperboxes (boxes in feature space) that people can read, but they cannot explain a single decision or say when a decision should not be trusted. We propose PACE (Prototype-aware, Additive, Contrastive Explanations), a set of explanations computed directly from the boxes of a centroid-based fuzzy min-max network (CFMNN), with no random sampling.

Our main explainer, PACE-SHAP, gives each feature its Shapley value for the model's winning margin: the same values KernelSHAP estimates by sampling. Because CFMNN's box scores add up feature by feature, PACE-SHAP computes these values exactly for up to 12 features. They equal those of the shap library's general exact explainer to rounding error, and come 16–30× faster. Above 12 features it samples, with a 1–2% error against exact values. With the same rows its values never change between runs, and in a test planned in advance it was more stable than KernelSHAP on 10 of 11 datasets even when both drew new background rows. It was also never significantly less faithful than KernelSHAP. Against KernelSHAP's usual setting it was more faithful by deletion on 6 of 11 datasets, mostly because it uses more background rows and every feature. When KernelSHAP was given the same rows and every feature, PACE-SHAP was more faithful by deletion on 3 of 13 datasets, level on the other 10 and worse on none, as expected when both target the same values, and KernelSHAP took 3.5–10× as long. The same exact computation also gives exact pairwise interaction values for up to 12 features. A second explainer, PACE-flip, finds which single-feature changes would change the decision. It gives brute-force occlusion's answer (up to rounding), the same on every run, in 0.05–1.4 ms per explanation: 25–134× faster than occlusion run in one batch. We also test trust signals for flagging predictions that are likely to be wrong. The model's own signals beat its plain margin but not standard methods that use the training labels. Adding CFMNN copies trained in other orders to those methods helps on most datasets, but copies trained on bootstrap samples help as much.

The explained model has a cost: on all 13 datasets, CFMNN is 0.2–21.5 points less accurate than the better of nearest neighbours and a Random Forest. In a test planned in advance, an explainable boosting machine was also more accurate on 12 of 13 datasets, and a decision tree on 10. Choosing θ by validation helped on 7, but with up to 23 times as many boxes. We find why it errs and fix it safely. On all 13 datasets, every training error of CFMNN we checked is a tie between a box of the right class and a box of the wrong class,

so shrinking the wrong box can fix it with no new labels. In a test planned in advance (6 datasets, 10 seeds), this self-repair raised macro-F1 on all 6, by 1.5–6.0 points, winning back 10–83% of the gap to a Random Forest. In a second planned test, it did not match choosing θ by validation, which stayed more accurate on 7 of 13 datasets, but it added no boxes. Buying up to 300 labels chosen by the order ensemble, plus the gate's labelled acceptance rows, added a further 0.7–2.0 points on 3 datasets; retraining CFMNN on the same labels helped significantly only once, in a post hoc test. Because each edit changes a single box, we can find the inputs whose prediction it can change: in 46,153 edits, no prediction changed outside that set, though the set is loose and does not bound changes to explanations. Of the predictions repair changed, 64–94% were corrections, against 41–52% for retraining. Letting an explanation choose where to cut the box did no better than cutting the thinnest slice.

# 1. Introduction

A fuzzy min-max neural network (FMNN) learns a set of boxes for each class, and a new point gets the class of the box it fits best. The centroid-based version (CFMNN) also stores the centre of each box and uses it to break ties. Each box is a readable rule, but a trained model holds tens to thousands of overlapping boxes, so the boxes alone do not tell a user why one point got its class.

Users need answers to two questions: **why** was this class chosen, and **when** should the prediction not be trusted? General-purpose explainers such as LIME and SHAP can answer the first question for any model, but they test the model on many random variations of each input. That makes them slow, and their answers change between runs.

Once a wrong prediction is found, a user also needs to **fix** the model without silently changing other predictions. Retraining CFMNN does not do this well: it depends on the order of the training rows, so retraining with a few more labels can change many predictions at once.

**Our idea.** A CFMNN box scores a point by averaging one score per feature. So when one feature changes, every box's score changes by an amount that depends on that feature alone. We can therefore work out exactly what the model would do under any single-feature change, without re-running it. All PACE explanations are built on this. The same structure limits what an edit to one box can change, which lets us repair the model and state which inputs each repair can affect. We do not claim that CFMNN is more accurate than other models. In our base run it is less accurate than the better of nearest neighbours and a Random Forest on every dataset, and in a later test an explainable boosting machine was more accurate on 12 of 13. Our aim is to explain, and safely fix, a model whose rules people can read.

**Research questions**

- **RQ1:** Are PACE's explanations exact, faithful to the model, and faster than existing methods?
- **RQ2:** Can signals built on PACE flag predictions that are likely to be wrong?
- **RQ3:** With every label counted, does buying new labels improve CFMNN beyond repairing its own training errors?
- **RQ4:** Can we find, for each edit, a set of inputs that contains every prediction it changes, and are more of repair's changes corrections than retraining's?

**Contributions**

1. **PACE-SHAP:** each feature's Shapley value for CFMNN's winning margin, computed from the model's structure instead of estimated by sampling: exact for up to 12 features, and from a fixed-seed sample of feature orders beyond that. Its exact values equal those of the shap library's general exact explainer, come 16–30× faster, and do not change between runs. Because they are the values KernelSHAP estimates, PACE-SHAP is never significantly less faithful than KernelSHAP; given the same rows, it is more faithful on 3 of 13 datasets, level on the rest, and 3.5–10× faster.
2. **Why CFMNN errs, and self-repair with certificates:** the finding that every CFMNN training error we checked is a tie between boxes of two classes, a gated shrink that fixes these errors with no new labels, and, for each box edit, a set of inputs found without labels that contains every prediction the edit changes. Both were tested in a run planned in advance against active learning and retraining.
3. **PACE-flip:** an exact, fast and repeatable way to find which single-feature changes would change a decision. To rank features for one prediction, PACE-SHAP is more faithful.
4. **A family of CFMNN-native explanations:** "why A, not B" explanations, a per-feature score breakdown, counterfactuals ("what is the smallest change that gives class B?"), including a sparse variant that changes as few features as it can, overall feature importance, and pairwise interaction values, exact for up to 12 features.
5. Trust signals for CFMNN, tested in runs planned in advance: the model's own signals lose to standard methods that use the training labels, and CFMNN copies trained in other orders add to those methods, as copies trained on bootstrap samples do.
6. **A tested re-implementation of CFMNN,** with two fixes to its training rule that we report separately from PACE.

## 2. Background

**Fuzzy min-max networks.** Simpson's FMNN (1992) learns boxes in one pass over the data: a point grows the nearest box of its class if the box stays under a size limit, otherwise it starts a new box. Many later versions changed how boxes are chosen, pruned or turned

into rules.

**CFMNN.** Aadarsh and Sai Prasad (PReMI 2023) added a centroid to each box, used it to break ties, and trained an ensemble of models on Spark. We use a single CFMNN and add explanations and trust signals to it.

**Existing explanation methods.** LIME fits a simple linear model around each input. KernelSHAP estimates each feature's share of the prediction (its Shapley value) by sampling. Both treat the model as a black box. Occlusion replaces one feature at a time and watches the prediction; it is exact but slow, because it re-runs the model for every feature and replacement value.

**The gap.** As far as we have found, no existing method explains individual CFMNN decisions from the model's own boxes, and none tells a CFMNN user when to distrust a prediction.

## 3. How CFMNN works

All features are scaled to the range 0 to 1. Each box has a lower and an upper corner, a class, and a centroid (the mean of the training points it absorbed).

- **Membership.** A point's membership in a box is the average of one score per feature. A feature's score is 1 when the point is inside the box on that feature, and falls towards 0.5 as the point moves outside.
- **Prediction.** The box with the highest membership wins. If several boxes tie, the one whose centroid is nearest wins. The winning margin that PACE-SHAP explains (Section 4) ignores this tie-break, so it is 0 for a tied point.
- **Training.** One pass over the data. A point grows the best box of its class if the box stays under a size limit; otherwise it starts a new box.

**Membership of a point in a box** = (score on feature 1 + score on feature 2 + … + score on feature d) ÷ d, where d is the number of features.

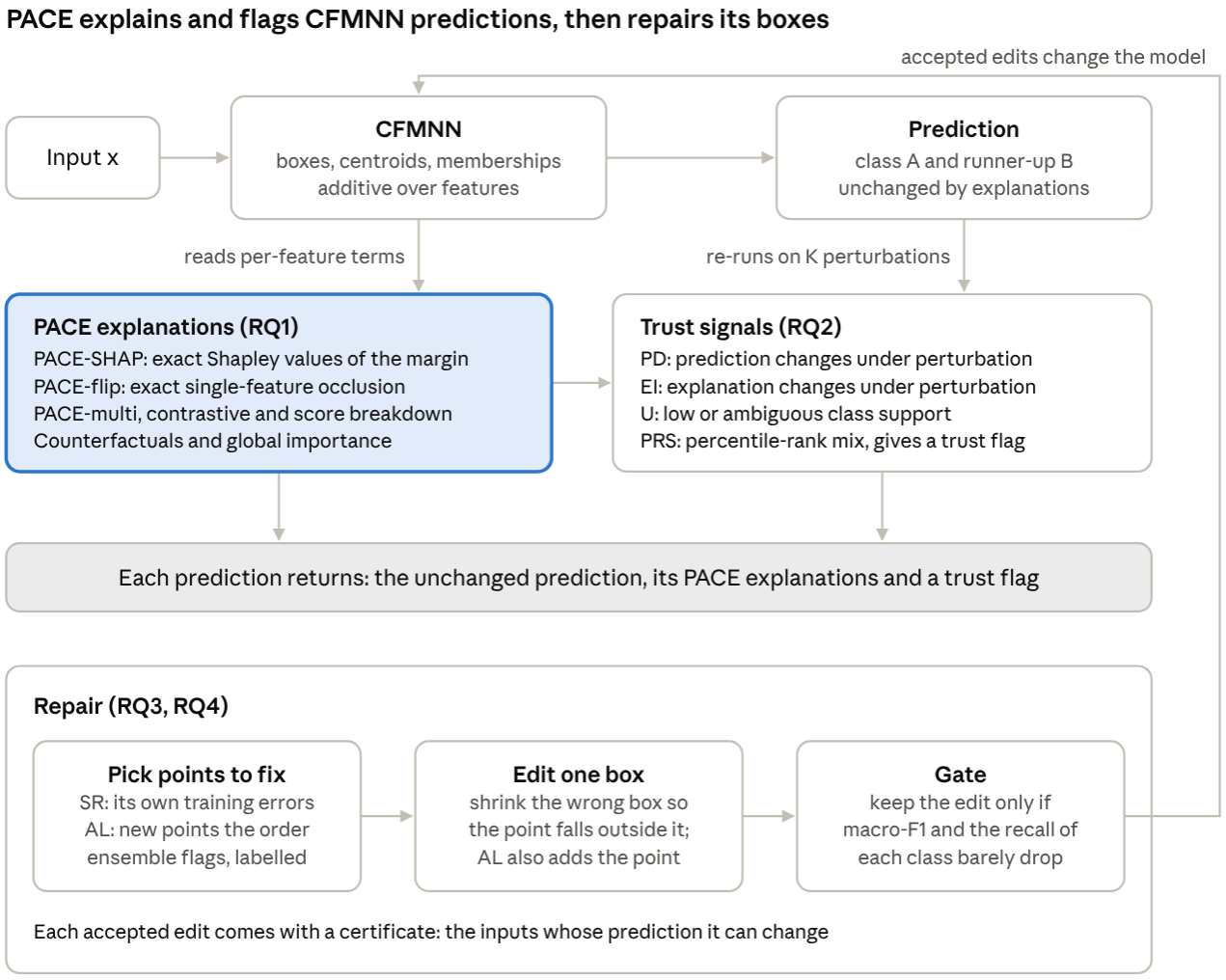
**Two fixes we found.** While re-implementing CFMNN from the paper's pseudocode, we found two training rules that hurt accuracy. We keep the paper's version too and report each fix as a separate model variant, so no gain is credited to PACE.

- **Centroid fix:** the base paper's text defines a box's centroid over all the training points in the box, but its pseudocode (Algorithm 2) leaves out points that land inside an existing box; we follow the text. Counting them raises Shuttle accuracy from 93.3% to 99.8%.
- **Expansion search:** the paper only tries to grow the single best box. Trying the next best box when that one is full helps on data with many features (Digits 81.7% → 95.3%), but hurts on 3-feature data (Skin 97.0% → 94.7%).

Our main model uses both fixes.

## 4. Our proposal: PACE explanations

PACE explains each prediction using only the model's own boxes. It never changes a prediction, and the one step that samples (PACE-SHAP with more than 12 features) uses a fixed seed, so the same call on the same points always gives the same answer (Figure 1).




PACE-CFMNN overview · explanations (RQ1), trust signals (RQ2) and repair (RQ3, RQ4)

*Figure 1. Overview. PACE reads CFMNN's per-feature membership terms and returns explanations next to the unchanged prediction. The trust signals re-run the frozen model on small perturbations. Repair is the only part that changes the model: it edits one box at a time, keeps an edit only if it passes the gate, and gives each accepted edit a certificate of the inputs whose prediction it can change.*

**The key property.** Changing feature i of a point to a new value z changes each box's membership by a known amount that depends only on feature i:

**New membership** = old membership + (new score on feature i − old score on feature i) ÷ d

The centroid distances used to break ties change in the same simple way. So the model's decision under any one-feature change can be worked out with two lookups per box.

**The PACE explanations**

| Explanation | Question it answers |
|---|---|
| PACE-SHAP (main) | How much does each feature add to the winning margin, shared out fairly over every combination of the other features? Equals the exact Shapley values of the margin for up to 12 features, and estimates them by sampling above that |
| PACE-flip | Which single-feature change would overturn this decision, and how much does each feature shrink the winning margin? |
| Contrastive | Why class A and not the runner-up class B, feature by feature? Uses the centroids when boxes tie |
| PACE-multi | Why class A and not its few closest competing boxes? |
| Score breakdown | How does each feature move the winning box's membership away from typical? Equals the exact Shapley values of that box's score |
| Counterfactual | What is the smallest change, confirmed by the model, that gives class B? Features such as age can be locked |
| Global importance | Which features matter most overall? |

**Why PACE-flip is fast.** PACE-flip replaces each feature in turn with 32 typical values and checks whether the predicted class changes. Done naively, this is occlusion: one full model run per feature and value. PACE-flip gets the same result in three steps:

1. It builds a lookup table of every box's feature scores at the 32 values, once per model.
2. It skips boxes that provably cannot win: 71–98% of boxes on all 11 datasets.
3. It processes all points and features together.

**How PACE-SHAP is exact.** A feature's Shapley value is how much it adds to the margin, averaged fairly over every combination of the other features; features left out take their values from a background row of training data. In a CFMNN, swapping one feature

changes each box's membership by a fixed amount that does not depend on the other features. So PACE-SHAP gets the margin of any combination by adding up these amounts, without re-running the model:

1. With up to 12 features it goes through every combination (at most 4,096) and gives the exact Shapley values.
2. With more features it samples 16 orders of the features per background row and adds each one's reverse (32 orders in all), from a fixed seed. The values still add up exactly to the margin's difference from the background average.
3. It averages over 200 background rows and skips boxes that can never set the margin under any combination.

**A small example.** Take two features and two boxes. Box 1 (class A) spans 0.2–0.4 on feature 0 and 0.2–0.5 on feature 1; box 2 (class B) spans 0.5–0.8 and 0.1–0.4. The point (0.43, 0.30) is just outside both boxes on feature 0 and inside both on feature 1. Its memberships are 0.97 for A and 0.93 for B, so A wins.

| Change | Membership in A's box | Membership in B's box | Predicted class |
|---|---|---|---|
| Feature 0 → 0.30 | 1.00 | 0.80 | A |
| Feature 0 → 0.70 | 0.75 | 1.00 | B |
| Feature 1 → 0.05 | 0.82 | 0.88 | B |
| Feature 1 → 0.90 | 0.72 | 0.68 | A |

The contrastive explanation says A leads only because of feature 0. PACE-flip shows that a change in either feature can overturn the decision, with feature 0 shrinking the margin slightly more. The two answer different questions, so we report both.

## 5. Trust signals

Three signals estimate whether a prediction should be trusted. They only rank predictions from most to least suspicious; they are not probabilities of error.

| Signal | What it measures |
|---|---|
| PD (prediction disagreement) | How often the prediction changes when the input is nudged slightly (50 small random nudges) |
| EI (explanation instability) | How much the "why A, not B" explanation changes under the same nudges |

| Signal | What it measures |
|---|---|
| U (uncertainty) | How weak the winning class's support is, and how close the runner-up class is |

The three are combined into one score, the PACE Refinement Score (PRS), by averaging their percentile ranks. The key test is whether EI adds anything beyond PD and U. When we repair the model (Section 6), the order ensemble, a CFMNN-based detector tested in Section 7, chooses which unlabelled points to label.

**A correction to U.** When boxes of two classes give a point the same highest membership (a tie), as when it lies inside boxes of both, the planned U formula scored it as less uncertain than most other points, although tied points are far more often wrong than other points. We therefore also test U*, which ranks every tie first and orders ties by how close the point is to each box's centroid. U* was designed after seeing the first results, so we fixed its definition and tested it again in a confirmatory run planned in advance (Section 7).

## 6. Repairing the model

On every dataset in our repair tests, each training row that CFMNN gets wrong is a tie. The row lies inside a box of its own class and inside a box of the wrong class, with membership 1 in both, and the nearest-centroid rule picks the wrong one. This held for 100% of training errors on all 6 datasets, at seeds 0 and 10. In the later test of repair at the θ chosen by validation, it held for all 347,339 training errors on all 13 datasets, at both θ, over 20 seeds. Simpson's network removed such overlaps with a contraction step; CFMNN has none. So the model can repair many of its own errors without new labels.

**The edit.** To fix a wrong prediction at a point inside the winning wrong-class box, we shrink that box on one feature: its face nearer the point moves 0.001 past the point. We cut on the feature where this removes the smallest share of the box's width. The box keeps only the training rows still inside it, and a box left empty is removed. Our first plan chose this feature with PACE's contrastive explanation; Section 7 reports that test.

**The gate.** Each edit is checked on labelled rows and undone if it fails. Compared with the model before the edit, macro-F1 may fall by at most 0.5 points and no class's recall by more than 1 point, and the model may hold at most 1.1 times its starting number of boxes.

**Three ways to repair.** All three use the same edit and gate, and try the same number of points: 300, or a quarter of the unlabelled pool if that is smaller.

| Method | Points it tries to fix | Gate rows | Labels beyond the training rows |
|---|---|---|---|
| Self-repair (SR) | The model's own training errors, in a fixed random order; a row already right by its turn is skipped | Training rows | None |
| Active learning (AL) | Points from an unlabelled pool, chosen in 10 rounds by the order ensemble (Section 7). Each is added with CFMNN's training rule, after the shrink when it lies inside a wrong-class box | Separate labelled acceptance rows | The bought labels and the acceptance rows |
| SR then AL | Self-repair, then active learning on the repaired model | Training rows, then acceptance rows | The bought labels and the acceptance rows |

**Certificates.** A shrink changes or removes one box, and an add grows one box or creates one. So an edit can change the prediction at an input only if a changed box is among that input's tied best boxes, before or after the edit. Everywhere else, the best membership, the tied boxes and their centroids stay the same. The inputs where a changed box is among the best therefore form a certificate: a set that contains every prediction the edit changes. A tight certificate also drops the inputs whose best boxes, before and after, all belong to one class. Both are found from the model before and after the edit, without labels.

**Fix share.** To compare how targeted two updates are, we count the test predictions each one changes against the starting model: fixed (wrong to right) and broken (right to wrong). The fix share is fixed ÷ (fixed + broken).

## 7. Experiments

**Setup.**

- **Datasets:** 3 synthetic sets where we know which features matter (S1–S3); Wine, Breast Cancer and Digits from scikit-learn; and the paper's Thyroid, Shuttle and Skin datasets. We added four more UCI datasets: MAGIC Gamma Telescope, Letter Recognition, Landsat Satellite and Page Blocks. Thyroid, Shuttle, Skin and these four come from the UCI Machine Learning Repository (Kelly, Longjohn and Nottingham),

with all their rows and features. Where the repository gives separate training and test files (Thyroid, Shuttle and Landsat), we pool them before splitting. Scaling to the range 0 to 1 uses the minimum and maximum of the training rows, and test values outside that range are clipped. The explanation tests use 11 datasets: S1–S3, Wine, Breast Cancer, Thyroid, Shuttle and these four. For the trust signals we also added three versions of S2 (imbalanced classes, label noise, shifted test data). We left out the paper's two largest datasets, CoverType (581,012 rows, 54 attributes) and Crop mapping (325,834 rows, 175 attributes), to keep our repeated tests within what one laptop can run (see Settings). We dropped the paper's Sepsis dataset: its 3 features cannot predict the outcome, so the best possible accuracy equals always guessing "survives". With the paper's rules, our single CFMNN scored 48.65% on it (mean of 5 seeds), against 68.76% for the paper's ensemble and 92.65% for always guessing "survives".

- **Protocol:** the paper's 80/20 train/test split, repeated over several random seeds.
- **Explanation test:** for 50 test points per dataset and seed (36 for Wine), over 10 seeds, we remove features in the order each method ranks them, most supportive of the predicted class first, and measure how fast the class's margin collapses (deletion). We also start from a real data row and put the top features back (insertion). A faster collapse, or a faster recovery, means the ranking is more faithful. KernelSHAP and LIME explain the same margin. We compare PACE with LIME, KernelSHAP, brute-force occlusion and a random ranking. We ran this test twice. The first run (seeds 0–9) compared PACE-flip and the other PACE methods. Before the second run (seeds 30–39), which added PACE-SHAP, we wrote down its tests and pushed them to GitHub. That run used 10 new seeds and gave the explainers background rows kept apart from the rows used to fill in removed features. It also added two checks: a stronger KernelSHAP (2,048 samples, no feature selection) and PACE-SHAP limited to KernelSHAP's 20 background rows.

**Base model.** Our CFMNN with both fixes, the model PACE explains, is transparent but not the most accurate. On all 13 datasets (80/20 split, 5 seeds) it is less accurate than the better of 10-nearest neighbours and a Random Forest with 200 trees: by less than 3 points on Shuttle, Wine, Breast Cancer, Thyroid and Digits, by 5.2 and 7.4 points on Skin and Landsat, and by 9.5–21.5 points on S1–S3, Page Blocks, MAGIC and Letter. The table gives accuracy on the test rows (%, mean ± standard deviation over the 5 seeds); Gap is CFMNN minus the better of the other two.

| Dataset | Training rows | CFMNN | 10-NN | Random Forest | Gap |
|---|---|---|---|---|---|
| S1 | 4,000 | 89.6 ± 1.1 | 95.1 ± 0.8 | 99.6 ± 0.1 | −10.0 |
| S2 | 4,000 | 80.6 ± 1.6 | 85.8 ± 1.8 | 90.1 ± 1.3 | −9.5 |
| S3 | 4,000 | 87.0 ± 0.8 | 88.7 ± 0.7 | 100.0 ± 0.0 | −13.0 |

| Dataset | Training rows | CFMNN | 10-NN | Random Forest | Gap |
|---|---|---|---|---|---|
| Wine | 142 | 98.3 ± 2.5 | 95.6 ± 3.2 | 100.0 ± 0.0 | −1.7 |
| Breast Cancer | 455 | 95.4 ± 1.1 | 97.4 ± 1.5 | 97.0 ± 1.0 | −1.9 |
| Digits | 1,437 | 95.3 ± 0.5 | 98.0 ± 0.7 | 97.6 ± 0.2 | −2.7 |
| Thyroid | 5,760 | 97.2 ± 0.6 | 94.2 ± 0.2 | 99.6 ± 0.1 | −2.5 |
| Shuttle | 46,400 | 99.8 ± 0.0 | 99.8 ± 0.0 | 100.0 ± 0.0 | −0.2 |
| Skin | 196,045 | 94.7 ± 2.1 | 99.9 ± 0.0 | 100.0 ± 0.0 | −5.2 |
| MAGIC | 15,216 | 72.5 ± 0.7 | 83.6 ± 0.4 | 88.3 ± 0.8 | −15.8 |
| Letter | 16,000 | 74.6 ± 0.7 | 94.2 ± 0.4 | 96.2 ± 0.2 | −21.5 |
| Landsat | 5,148 | 84.3 ± 1.1 | 89.8 ± 0.4 | 91.7 ± 0.9 | −7.4 |
| Page Blocks | 4,378 | 86.3 ± 5.6 | 94.8 ± 0.4 | 97.3 ± 0.6 | −11.0 |

With the paper's rules, our single CFMNN comes within 2 points of the paper's ensemble on Thyroid, Shuttle and Skin:

| Dataset | Ours, paper's rules (%) | Ours, with both fixes (%) | Paper's ensemble (%) |
|---|---|---|---|
| Thyroid | 95.0 | 97.2 | 96.9 |
| Shuttle | 93.3 | 99.8 | 94.0 |
| Skin | 96.9 | 94.7 | 97.6 |

**Against other readable models.** CFMNN is not the most accurate model people can read either; we study it for its structure, which makes PACE exact. In a test planned in advance (seeds 70–89, the same split), we compared it with a decision tree (CART; Breiman, Friedman, Olshen and Stone), logistic regression, an explainable boosting machine (EBM; Lou, Caruana and Gehrke; Nori, Jenkins, Koch and Caruana) and GFMM (Gabrys and Bargiela). CFMNN, the tree and logistic regression chose their setting by 3-fold cross-validation on the training rows (θ from 0.5 to 0.05; 4–1,024 leaves; C from 0.01 to 100), and we also kept CFMNN at θ = 0.3. EBM ran with main effects only (one curve per feature) and GFMM at θ = 0.3, both untuned, because the full versions took too long; this was fixed before the run. Each count is of datasets, from paired Wilcoxon tests over the 20 seeds with Holm correction across the 13.

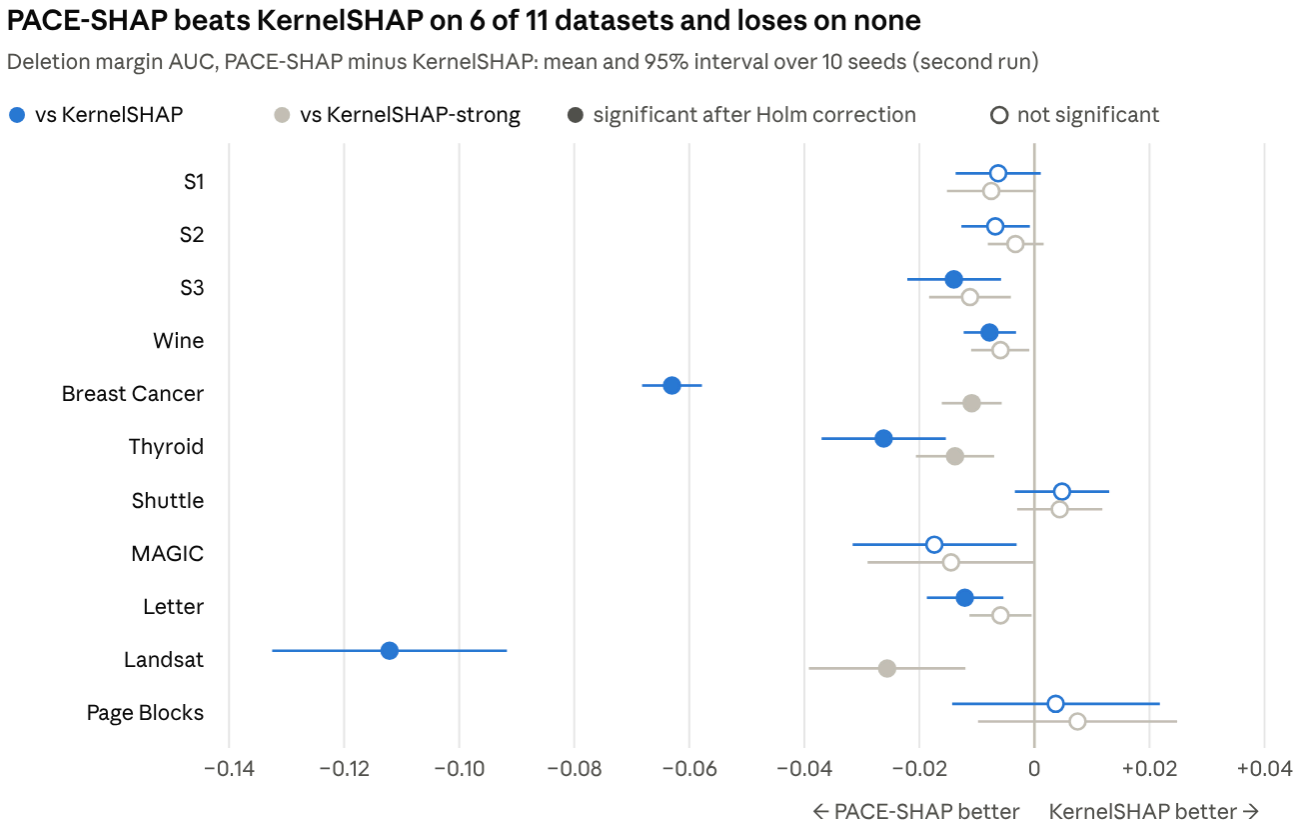
| CFMNN against | CFMNN more accurate | Level | CFMNN less accurate |
|---|---|---|---|
| Tree, both tuned | 5 | 0 | 8 |
| Logistic regression, both tuned | 7 | 2 | 4 |

| CFMNN against | CFMNN more accurate | Level | CFMNN less accurate |
|---|---|---|---|
| EBM, CFMNN tuned | 2 | 2 | 9 |
| Tree, CFMNN at θ = 0.3 | 3 | 0 | 10 |
| Logistic regression, CFMNN at θ = 0.3 | 4 | 2 | 7 |
| EBM, CFMNN at θ = 0.3 | 0 | 1 | 12 |
| GFMM, both at θ = 0.3 | 4 | 3 | 6 |

- **EBM was the strongest.** At θ = 0.3, CFMNN was level with it only on Wine, and up to 14.2 points less accurate elsewhere. Tuned, CFMNN beat it only on Skin and Letter.
- **θ = 0.3 was not the most accurate choice.** θ chosen by validation was more accurate on 7 datasets, by 0.2–20.0 points (Letter 20.0, Page Blocks 10.3, MAGIC 10.1), and never less accurate. On 6 of these it chose θ = 0.05, the smallest value tried, with 6–23 times as many boxes (Letter 5,325 against 565). Self-repair at θ = 0.3 did not make up for this. In a second test planned in advance, on the same seeds and split, θ chosen by validation without repair was more accurate than repair at θ = 0.3 on 7 datasets, by up to 17.5 points (Letter); repair at θ = 0.3 was more accurate only on Thyroid. With repair at both θ, the chosen θ was more accurate on 8 datasets and never less. At the chosen θ, repair was more accurate on 3 datasets, by at most 1.6 points, and never less; on Letter, MAGIC and Landsat that θ left only 2–11 training errors on average. Macro-F1 gave similar counts. Repair at θ = 0.3 was still more accurate than no repair on 7 datasets and added no box, so the reason to repair at θ = 0.3 is the smaller model, not accuracy. Every other result in this paper keeps θ = 0.3.
- **Trees were also smaller.** The tuned tree had fewer leaves than CFMNN at θ = 0.3 had boxes on 10 of 13 datasets. A leaf tests 2.3–9.3 features on average; a box limits 79–100% of them.
- **The reference models stay ahead.** Tuned CFMNN trailed the better of a tuned nearest-neighbour classifier and the Random Forest on every dataset, by up to 11.9 points. At θ = 0.3 it tied the Random Forest on Wine (98.3%) and trailed elsewhere by 0.2–21.3 points.
- **Checks.** By macro-F1 the counts were the same for the tree and logistic regression, and similar for EBM (2, 3, 8) and GFMM (4, 5, 4). Several tuned models often chose the end of their grid (θ = 0.05, C = 100, k = 1), so wider grids might change some counts.

**Faithfulness.** In the second run, PACE-SHAP is never significantly less faithful than any method tested, and more faithful than each of them on several datasets (Figure 2). By deletion, the main planned test, it is more faithful than KernelSHAP on 6 of 11 datasets (S3,

Wine, Breast Cancer, Thyroid, Letter and Landsat), by insertion on 7, and never significantly less faithful. Against the stronger KernelSHAP it wins on 3 datasets by deletion (Breast Cancer, Thyroid, Landsat) and 4 by insertion, and is never significantly less faithful. Measured against KernelSHAP's own lead over a random ranking, PACE-SHAP's lead by deletion is small on S1–S3, Wine and Letter (2–4%), moderate on MAGIC, Thyroid and Breast Cancer (10–28%) and large on Landsat (67%), with paired effect sizes (d_z) of 0.6 to 8.6; on Shuttle and Page Blocks KernelSHAP is slightly ahead, not significantly. The table after Figure 2 gives the mean AUCs.




pace-cfmnn/results/faithfulness_v3/summary.md · pre-registered second run, seeds 30–39 · 11 datasets · paired Wilcoxon test, Holm correction

*Figure 2. PACE-SHAP minus KernelSHAP in deletion-margin AUC per dataset, second run: mean and 95% interval over 10 seeds. Left of zero means PACE-SHAP is more faithful. Blue: against KernelSHAP as in the first run (20 background rows, the shap library's default feature selection); grey: against a stronger KernelSHAP (2,048 samples, no feature selection). Filled: significant after Holm correction across the 11 datasets ($p < 0.05$). The intervals are not adjusted for the 11 datasets, so a few exclude zero although the corrected test is not significant: S2 and MAGIC in blue, and S3, Wine, MAGIC and Letter in grey.*

Mean deletion-margin AUC in the second run (seeds 30–39), over all explained points and seeds; lower is more faithful. Values can be negative because the margin falls below zero once the decision changes. Bold marks the lowest value in each row; it is not a significance test (Figure 2 gives the tests).

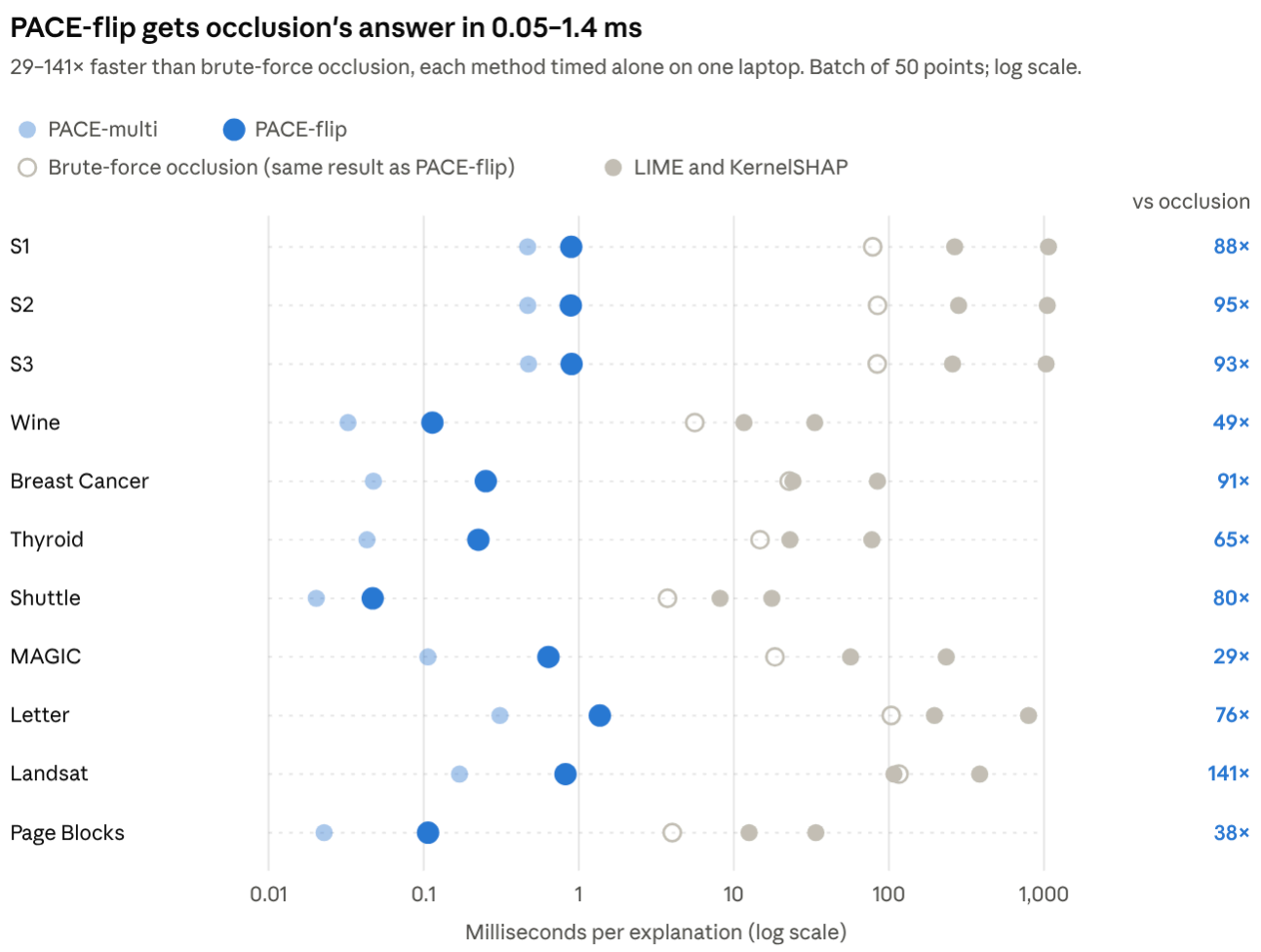
| Dataset | PACE-SHAP | KernelSHAP | Stronger KernelSHAP | LIME | PACE-flip | Random ranking |
|---|---|---|---|---|---|---|
| S1 | **−0.1389** | −0.1326 | −0.1314 | −0.1095 | −0.0892 | 0.2708 |
| S2 | **−0.1317** | −0.1249 | −0.1285 | −0.0951 | −0.0755 | 0.2565 |
| S3 | **−0.0552** | −0.0412 | −0.0440 | 0.0117 | 0.0057 | 0.3179 |
| Wine | **−0.0761** | −0.0684 | −0.0702 | −0.0387 | −0.0545 | 0.2356 |
| Breast Cancer | **0.1114** | 0.1743 | 0.1222 | 0.1412 | 0.1366 | 0.3981 |
| Thyroid | 0.5486 | 0.5748 | 0.5624 | 0.5708 | **0.5440** | 0.7392 |
| Shuttle | 0.3701 | **0.3652** | 0.3657 | 0.3812 | 0.3741 | 0.5516 |
| MAGIC | **−0.0522** | −0.0348 | −0.0377 | −0.0115 | 0.0153 | 0.1400 |
| Letter | **−0.6897** | −0.6776 | −0.6838 | −0.6561 | −0.6441 | −0.3811 |
| Landsat | **−0.3287** | −0.2166 | −0.3031 | −0.2954 | −0.2546 | −0.0497 |
| Page Blocks | 0.0501 | 0.0464 | **0.0426** | 0.0487 | 0.0514 | 0.1282 |

- PACE-SHAP and KernelSHAP aim at the same Shapley values; PACE-SHAP computes them better. It has no sampling error up to 12 features (and a 1–2% error above), keeps every feature (by default the shap library keeps at most 10 and gives the rest zero, which hurts most on Breast Cancer and Landsat, with 30 and 36 features), and uses 200 background rows instead of 20. The last two matter most. The conditional-replacement run's plan also compared the two on equal terms, with KernelSHAP using the same 200 rows, every feature and 200 coalitions, on the usual (not conditional) tests. There PACE-SHAP was more faithful on 3 of 13 datasets by deletion (S3, Breast Cancer, Landsat) and 2 by insertion (Breast Cancer, Landsat), level on the rest, and never significantly less faithful, while KernelSHAP took 3.5–10× as long on every dataset (timed while other runs shared the laptop). Limited to KernelSHAP's 20 rows, PACE-SHAP is still never significantly less faithful than KernelSHAP.
- PACE-SHAP is more faithful than PACE-flip on 8 of 11 datasets by both tests, and level on Thyroid, Shuttle and Page Blocks. In the first run KernelSHAP beat PACE-flip on 6 datasets (S1–S3, Wine, MAGIC, Letter) and PACE-flip won on 3 (Breast Cancer, Thyroid, and Landsat by deletion). PACE-flip trails because of what it measures: it changes one feature at a time and ranks features first by how often that change alone flips the class. CFMNN decides by its best-matching box, and after a single change another box of the same class can take over, so features that matter mainly in combination get little credit. KernelSHAP and PACE-SHAP average over combinations of features. We test PACE-flip because its scores are easy to read as a ranking, but to rank features for one prediction we propose PACE-SHAP; PACE-flip answers which single change would overturn a decision. Brute-force occlusion gives PACE-flip's result up to floating-point rounding; PACE-flip's gain is speed.

- **Stability:** with a fixed seed, every PACE method gives identical explanations on every run. With a new seed, PACE-SHAP's sampled mode (more than 12 features) keeps 0.94–0.99 of its top 3 features; LIME keeps 0.52–0.90 and KernelSHAP 0.15–0.73. Part of that gap came from KernelSHAP drawing new background rows on each run: in an analysis made after the run, PACE-SHAP limited to 20 newly drawn rows was more stable than KernelSHAP on only 5 of 11 datasets. A test planned in advance, on 10 new seeds, separated the two causes. PACE-SHAP was more stable than KernelSHAP on all 11 datasets when each method kept its background rows, and on 10 of 11 (level on Page Blocks) when every method got new ones.
- On the synthetic data, LIME names the truly relevant features most often (0.89–0.92 of its top 3), ahead of PACE-SHAP (0.79–0.85), KernelSHAP (0.72–0.81) and PACE-flip (0.49–0.55). Yet LIME is less faithful to the model than PACE-SHAP on 9 of 11 datasets: naming the data's relevant features is not the same as explaining what this model does. PACE-flip reports which single-feature changes move this model's decision, and only 42–47% of those changes involve the relevant features.

**Ties.** For many explained points the two strongest classes have the same score, so the margin is 0: in the second run, 79% of points on Page Blocks, 75% on MAGIC, 55% on Letter, 25% on Landsat and 11% on Shuttle. CFMNN then predicts the class whose centroid is nearest, which the margin ignores, and dividing a curve by a margin of 0 keeps little more than its signs. A test planned in advance, on 10 new seeds, therefore also compared raw margin curves, not divided by each point's margin, and untied points alone. On the raw curves, PACE-SHAP was more faithful than KernelSHAP on 10 of 11 datasets by deletion (level on Page Blocks) and on all 11 by insertion. On the untied points of the 9 datasets with at least 10 of them in every seed, it was more faithful on 7 by deletion and 6 by insertion. It was never significantly less faithful. In the same test, LIME given only the 200 background rows, as the second run's plan said, was less faithful than PACE-SHAP on 9 of 11 datasets by deletion and on all 11 by insertion.

**Speed.** PACE-flip takes 0.05–1.4 ms per explanation (Figure 3).




pace-cfmnn/results/faithfulness/timing.csv · each method timed alone, mean over 3 seeds, batch of 50 points (36 for Wine), Intel i7-8650U

*Figure 3. Time per explanation (log scale) and PACE-flip's speed-up over brute-force occlusion, which returns the same ranking.*

- That is 29–141× faster than occlusion, which computes the same thing, 89–316× faster than LIME and 292–1,194× faster than KernelSHAP, each timed alone on the same laptop. This occlusion calls the model once per point and feature. Sending all points in one call saves little (at most 2.2×), and PACE-flip is still 25–134× faster than that batched version (3 seeds, a check made after the run).
- PACE-multi is faster still when many points are explained together.
- PACE-SHAP does not save time against KernelSHAP's estimates. Timed alone, it took 1.2–3.0× as long as KernelSHAP on 9 of 11 datasets and less time on Wine and Shuttle. Limited to KernelSHAP's 20 background rows, it is 4.3–14× faster than KernelSHAP and still never significantly less faithful. Against general methods that compute the same exact values, it is fast. On the 6 datasets with at most 12 features (3 seeds, 10 points each), its values equal those of the shap library's ExactExplainer and of KernelSHAP given every coalition to within 3.3e-16, and it is 16–30× faster per explanation than ExactExplainer: for example 2.5 s against 53.7 s on S1, and 19 ms

against 478 ms on Shuttle. Above 12 features, its sampled mode with 32 orders per row is within 1.0% (Wine) and 1.7% (Letter) of the exact values in relative L1 error, with top-3 overlaps of 0.995 and 1.0. Exact values on Letter take 41.7 s per explanation, against 0.9 s for the sampled mode.

**Conditional replacement.** To remove a feature, PACE-SHAP and KernelSHAP fill in values from other rows. Taken from any row, as above, these can form inputs unlike real data. A conditional version takes them from rows that resemble the point on the features it keeps (Gaussian weights as in Aas, Jullum and Løland, but on distances scaled by each feature's spread instead of the full covariance, $\sigma = 0.1$); we tested it in a second run planned in advance (13 datasets, 10 new seeds).

- Its inputs are far more realistic. We count an input as off the data when it is farther from its nearest training row than 95% of training rows are from theirs. Outside Shuttle, the conditional game's inputs were off at most 10% of the time, against up to 67% for the usual game. On Shuttle, whose rows are packed very densely, the two were off 57% and 71% of the time. Part of this comes from copying a few training rows.
- Like for like, on a test that also fills in removed features conditionally, the conditional PACE-SHAP was more faithful than the conditional KernelSHAP on 6 of 13 datasets, by deletion and by insertion, and never significantly less faithful.
- But the conditional explainers were rarely more faithful than the usual ones, even on that test: PACE-SHAP beat its conditional version there on 10 of 13 datasets by deletion and 9 by insertion. The one exception was Shuttle, where the conditional version was more faithful by deletion (AUC 0.452 against 0.482, in 9 of 10 seeds). They also gave a noisy copy of a relevant feature less credit, not more as we expected. A likely reason is that at $\sigma = 0.1$ the conditional game rests on about one row once half the features are kept, on 7 of the 13 datasets. Two checks made after the run, on up to 6 datasets and 5 seeds, support this. With wider weights, which average over more rows, the conditional PACE-SHAP nearly matched PACE-SHAP on the synthetic data at $\sigma = 1$. At $\sigma = 0.5$ and 1 it was slightly more faithful than PACE-SHAP on the conditional test on Shuttle and MAGIC, and gave the noisy copy slightly more credit, as we expected. The price is realism: on MAGIC its inputs were off the data 5% of the time at $\sigma = 0.5$ and 12% at $\sigma = 1$, against 2% at $\sigma = 0.1$ and 20% for the usual game. Keeping $\sigma = 0.1$ but using four times as many rows instead halved its loss on S1, whose independent features make the two games the same in theory, and changed little on the real datasets.

**Trust signals (RQ2).** We ran this test twice. The first run, over 10 random seeds, is where we found the U* correction. Before the second run, which we call the confirmatory run (seeds 10–19), we wrote down every definition and test and pushed them to GitHub. That run used 10 fresh seeds, the 4 new UCI datasets and three standard methods for

comparison. In both runs the model trains on half of each dataset, the scores are calibrated on 10%, and we check on the remaining 40% (in the confirmatory run, at most 20,000 rows) how well each score ranks the model's wrong predictions first. In the confirmatory run, datasets under 2,000 rows use 5-fold cross-validation instead. A dataset is tested only if it gives at least 50 errors in every seed. That rules out Wine, Breast Cancer and Shuttle, which have as few as 3, 20 and 39 errors in a seed. The 6 synthetic sets come from one generator, so they are averaged and tested as one unit.

The three standard methods all use the training labels, which PACE's signals do not. Trust Score compares a point's distance to training points of the predicted class with its distance to the other classes. Nearest-neighbour disagreement is the share of the 10 nearest training points that have another label. The order ensemble is the share of 5 CFMNNs, trained on the same data in other orders, that disagree with the model. A difference counts as significant when a paired test across the 10 seeds, corrected for testing several units at once, gives p below 0.05. The table shows the confirmatory run's area under the precision–recall curve: higher is better, and a random ranking scores about the share of errors.

| Dataset | Share of errors | Plain margin | PD + U* | All three (PRS*) | Trust Score | Nearest-neighbour disagreement | Order ensemble |
|---|---|---|---|---|---|---|---|
| Synthetic (6 sets, averaged) | 0.17 | 0.39 | 0.41 | 0.35 | 0.55 | **0.69** | 0.65 |
| Digits | 0.04 | 0.39 | 0.40 | 0.19 | **0.86** | 0.82 | 0.60 |
| Thyroid | 0.03 | 0.26 | 0.23 | 0.23 | 0.11 | 0.09 | **0.43** |
| Skin | 0.05 | 0.22 | 0.42 | 0.38 | 0.98 | **0.99** | 0.75 |
| MAGIC | 0.28 | 0.33 | 0.44 | 0.40 | 0.69 | **0.72** | 0.59 |
| Letter | 0.26 | 0.41 | 0.59 | 0.56 | **0.97** | **0.97** | 0.78 |
| Landsat | 0.15 | 0.32 | 0.40 | 0.36 | **0.74** | 0.73 | 0.54 |
| Page Blocks | 0.14 | 0.19 | 0.44 | 0.37 | 0.85 | **0.86** | 0.62 |

- **PD + U* beats the plain margin** on 6 of the 8 units, in all 10 seeds each. The gain is largest where many points tie, such as Page Blocks (0.44 vs 0.19) and Skin (0.42 vs 0.22). It does not beat the margin on Digits (a small gain, not significant) or Thyroid (slightly worse).
- **EI does not help.** Adding it made the combined score significantly worse on 6 of the 8 units and never significantly better. On its own it is useful on some datasets, such as Thyroid and Skin, and close to random on others.
- **U* is confirmed.** It beats the planned U on all 6 real datasets where at least 1% of points tie, in all 10 seeds. The planned U is no better than random on MAGIC and Letter, and not significantly better on Page Blocks.

- **The standard methods are much better.** Trust Score and nearest-neighbour disagreement beat PD + U* on 7 of the 8 units, and the order ensemble beats it on all 8. A check made after the run suggests one reason for the first two: a plain 10-nearest-neighbour classifier is more accurate than CFMNN on 12 of the 13 datasets (for example 92% vs 74% on Letter), so they largely flag points where a more accurate model disagrees. Thyroid, the one dataset where CFMNN is more accurate, is also the one where PD + U* beats them. This does not explain the order ensemble, which is made of CFMNNs.

**The combination run (seeds 20–29).** A signal computed from one CFMNN cannot know what a more accurate model would predict, so we then asked whether CFMNN's own signals add to the standard ones. A simple learned model (a logistic regression) combined the signals; it was fitted on the 10% calibration rows, using their labels. We designed this test on the first 10 seeds, wrote it down and pushed it to GitHub, and then ran it on 10 fresh seeds. Page Blocks' share of errors is lower here than in the first table (0.10 against 0.14) only because the seeds differ: two seeds of the confirmatory run had error rates of 0.20 and 0.24. Area under the precision–recall curve:

| Dataset | Share of errors | Nearest-neighbour disagreement | Trust Score | Both combined | Both + order ensemble |
|---|---|---|---|---|---|
| Synthetic (6 sets, averaged) | 0.17 | 0.70 | 0.56 | 0.73 | **0.77** |
| Digits | 0.05 | 0.83 | **0.88** | 0.87 | 0.85 |
| Thyroid | 0.03 | 0.10 | 0.11 | 0.12 | **0.46** |
| Skin | 0.05 | **1.00** | 0.99 | **1.00** | **1.00** |
| MAGIC | 0.26 | 0.70 | 0.67 | 0.74 | **0.75** |
| Letter | 0.26 | 0.97 | 0.97 | **0.98** | **0.98** |
| Landsat | 0.15 | 0.73 | 0.74 | 0.78 | **0.79** |
| Page Blocks | 0.10 | 0.82 | 0.81 | **0.87** | **0.87** |

- **An ensemble of CFMNNs adds information.** Adding the order ensemble to the two standard methods improved error detection on 5 of the 8 units, in all 10 seeds each. The gain is largest on Thyroid (0.12 to 0.46), where CFMNN is more accurate than nearest neighbours. It made no significant difference on Skin or Page Blocks, and slightly lowered the result on Digits. A test planned in advance, on 10 new seeds, repeated this on 5 units (the synthetic family, Thyroid, MAGIC, Letter and Landsat; again lower on Digits). It also showed that the gain is not specific to CFMNN's sensitivity to order: 5 CFMNNs trained on bootstrap samples, in the model's own order, did as well on all 8 units, although the order ensemble still added to them on 4

(the synthetic family, Thyroid, MAGIC and Letter). An order ensemble of single-pass logistic regressions helped only on the synthetic family, where a linear model is far more accurate than CFMNN, and was slightly worse on Skin.

- **The strongest detector overall** is this combination. It beats nearest-neighbour disagreement on all 8 units and Trust Score on 7; only on Digits is another detector clearly better.
- **The geometry signals add little** once the standard methods are included: ties, U* and the distance to the box's nearest training point. In an analysis made after the run, adding them raised AUPRC significantly only on Letter, by 0.0015.

**Counterfactuals.** For 30 test points per dataset and seed, over 10 seeds (40–49), we asked each method for the smallest change that makes the model predict the runner-up class, and checked every answer on the model. On Thyroid, age and sex were locked. DiCE was asked for 5 candidates and we kept the nearest valid one, using DiCE's own sparsity step. The table uses the points where both methods found a valid answer (27–30 of 30 in each seed). Values are PACE / DiCE, averaged over seeds; time is the median per counterfactual, measured while several runs shared the CPU, so compare times within a row only. Bold marks a significant difference (a paired test over the 10 seeds, Holm correction across the 11 datasets, $p < 0.05$). The run was planned to compare PACE-sparse with DiCE (see Sparse counterfactuals); the comparison in this table is an analysis made after the run.

| Dataset | Size of change | Features changed | Distance to real data of the target class | Time (ms) |
|---|---|---|---|---|
| S1 | 0.39 / 0.41 | **1.58 / 1.32** | **1.16 / 1.23** | **7.0 / 6,784** |
| S2 | 0.36 / 0.39 | **1.56 / 1.32** | **1.11 / 1.19** | **7.2 / 6,996** |
| S3 | 0.37 / 0.39 | 1.44 / 1.36 | **1.14 / 1.24** | **6.4 / 6,700** |
| Wine | **0.70 / 0.80** | **3.44 / 2.35** | **1.32 / 1.67** | **4.0 / 560** |
| Breast Cancer | 1.43 / 1.66 | **10.96 / 4.51** | **1.68 / 3.01** | **9.5 / 1,243** |
| Thyroid | **0.16 / 0.29** | **1.90 / 1.60** | **0.25 / 0.51** | **3.0 / 961** |
| Shuttle | **0.12 / 0.25** | **3.29 / 1.37** | **0.15 / 0.28** | **7.6 / 1,045** |
| MAGIC | **0.25 / 0.33** | **2.56 / 1.41** | **0.26 / 0.46** | **7.8 / 2,966** |
| Letter | **0.21 / 0.32** | **2.66 / 1.45** | **0.72 / 0.92** | **30.8 / 10,622** |

| Dataset | Size of change | Features changed | Distance to real data of the target class | Time (ms) |
|---|---|---|---|---|
| Landsat | 1.02 / 1.10 | **10.64 / 3.32** | **1.69 / 2.53** | **24.4 / 3,767** |
| Page Blocks | 0.24 / 0.35 | **2.18 / 1.37** | **0.24 / 0.51** | **7.9 / 555** |

- The model confirms 99–100% of PACE's counterfactuals and 97–100% of DiCE's.
- PACE's counterfactuals land closer to real data of the target class on all 11 datasets, make a smaller change on 5 of the 8 real datasets (Wine, Thyroid, Shuttle, MAGIC and Letter) with no significant difference on the other 6, and are about 70–1,000× faster. DiCE changes fewer features on 10 of the 11 datasets (no significant difference on S3).
- Both keep the locked features. The nearest real example of the other class changes 6–30 features on average, more than twice as many as either method, and keeps Thyroid's locks in only 9% of cases.

**Sparse counterfactuals.** PACE-sparse looks for the fewest features to change, among moves into boxes of the target class. It scores each candidate set exactly from CFMNN's per-feature terms and checks its answer on the model. In the same run, in tests over seeds planned in advance, it changed 0.1–1.1 fewer features than DiCE on 10 of 11 datasets, and made a smaller change on all 11.

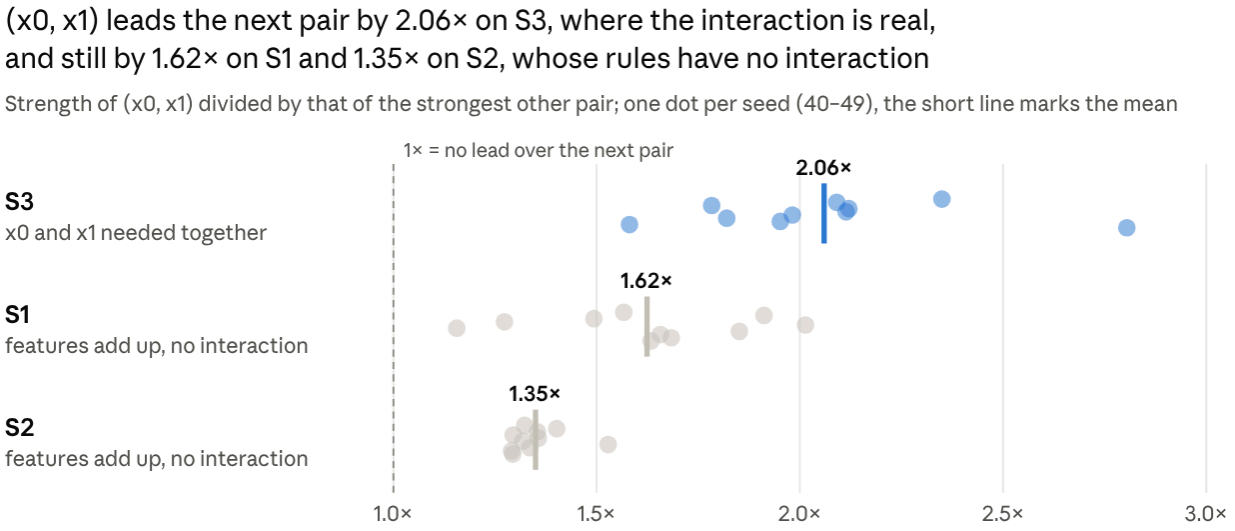
- On S1–S3, MAGIC, Letter and Page Blocks, it changed a single feature for 85–97% of points; DiCE did so for 56–68%.
- Shuttle is the exception: PACE-sparse changed 0.9 more features than DiCE. In a later check on one seed, every point where DiCE needed fewer features started inside a box of its own class. That box overlaps the target's boxes, so moving into a target box is not enough and a second feature is needed to leave it.
- Sparsity has a price. PACE-sparse lands closer to real data of the target class than DiCE on the 8 real datasets, but 5–55% farther than PACE on all 11. It takes 13 ms to 4.4 s per counterfactual: faster than DiCE on 10 of 11 datasets, but 3.5× slower on Breast Cancer.

**Global importance.** All rankings, PACE's and the standard ones, find the 3 relevant synthetic features. Over 10 seeds, PACE-flip averaged over points agrees with permutation importance more than KernelSHAP does on 10 of 11 datasets, significantly on 3 (Shuttle, Letter, Landsat); on MAGIC KernelSHAP is ahead, not significantly. Permutation importance changes one feature at a time, as PACE-flip does, so part of this agreement comes from how the two are built. PACE-flip takes at most 11 seconds per dataset on average (14.3 s in the slowest run), against up to about 3 minutes for KernelSHAP, both measured while several runs shared the laptop. PACE-flip averaged over every test point

and KernelSHAP over 50, which is what each could afford, so the comparison is not like for like: in a later check on Page Blocks with fresh seeds, limiting PACE-flip to the same 50 points removed about a quarter of its lead.

In the second run (10 new seeds), we also averaged PACE-SHAP over the same 50 points as KernelSHAP. This ranked features no better: neither agreed with permutation importance significantly more on any of the 11 datasets. Averaging probably smooths out most of KernelSHAP's sampling noise; we did not test this. PACE-flip's lead over KernelSHAP replicated on Letter and Landsat, appeared on Page Blocks and did not replicate on Shuttle.

**Interactions.** PACE-SHAP's table also gives pairwise interaction values, exact for up to 12 features: how much two features add together beyond what each adds alone. On S3, whose class needs x0 and x1 together, (x0, x1) was the strongest pair in all 10 seeds, as planned. But it was also the strongest on S1 and S2, whose rules just add features up, so the planned test was weak (Figure 4). Interaction values describe the model, not the data.




pace-cfmnn/results/extensions_v1/interactions/runs.csv · pre-registered run, seeds 40–49 · pair strength = mean |interaction| over 50 points; ratio as in posthoc_review.md

*Figure 4. How far (x0, x1) leads the next strongest pair, per seed. The lead is largest on S3, whose range overlaps S1's but not S2's. On S1 and S2 the lead comes from the model; a likely reason, not tested, is that its boxes approximate the sloped boundary as a staircase.*

**Repair, first run.** We first tested the repair questions as first posed, in a run planned in advance (seeds 10–19). It asked whether PACE's contrastive explanation can choose which feature of the wrong box to cut, and whether the order ensemble can choose which points to label. It used the 6 datasets with at least 2,000 rows and enough repairable points: Thyroid, Landsat, Page Blocks, MAGIC, Letter and Skin. Cutting on the explanation's top feature was no better than cutting the thinnest slice (the main test; no

significant difference on any dataset) or a random feature (worse on Letter). Shrinking with that cut beat only adding points on all 6 datasets, by 0.7–8.6 macro-F1 points, and the order ensemble beat random choice on 3.

**Why we changed the questions.** That run could not say where its gains came from. Exploratory checks on seeds 0–2 found three things. First, every training error is a tie (Section 6), and self-repair, with no new labels, was no more than half a point behind buying 300 labels on 5 of the 6 datasets. Second, the gate reads labelled acceptance rows, 548–24,506 per dataset, which the first run did not count. Third, retraining CFMNN on the same labels gained nothing. Trying every feature and keeping the cut the gate scored best gained only −0.6 to +1.1 points over the thinnest cut, so we closed the question of which feature to cut. We then wrote RQ3 and RQ4 as in Section 1 and tested them in a second run planned in advance (seeds 50–59), written down and pushed to GitHub before it started. It counts every label, and adds CFMNN retrained, 10-nearest neighbours and a Random Forest given the same labels.

**Repair, second run (RQ3).** The data are split into training rows (50%), an unlabelled pool (20%), acceptance rows for the gate (10%), test rows (10%) and rows held back (10%). Results are measured on the test rows after the last edit. Each comparison is a paired Wilcoxon test over the 10 seeds, with Holm correction across the 6 datasets. Tests of "no worse than" are one-sided, with margins of 1 macro-F1 point and 2 points of the lowest class recall. The table gives test macro-F1 (%, mean over 10 seeds). The methods that buy labels read 821–24,806 labels beyond the training rows: the budget plus the acceptance rows.

| Method | Thyroid | Landsat | Page Blocks | MAGIC | Letter | Skin |
|---|---|---|---|---|---|---|
| Starting model | 84.0 | 82.0 | 67.6 | 72.6 | 74.3 | 93.1 |
| Self-repair (no new labels) | 90.0 | 83.5 | 70.1 | 75.9 | 76.6 | 98.8 |
| Active learning | 90.3 | 83.9 | 72.4 | 77.7 | 76.3 | 99.2 |
| SR then AL | 90.5 | 84.5 | 72.4 | 77.9 | 77.3 | 99.3 |
| CFMNN retrained on the same labels, shuffled | 85.7 | 82.9 | 64.2 | 71.1 | 74.7 | 91.2 |
| 10-nearest neighbours, same labels | 58.4 | 87.9 | 68.2 | 80.2 | 93.5 | 99.9 |
| Random Forest, same labels | 97.7 | 89.6 | 82.4 | 86.8 | 95.9 | 99.9 |

- **Self-repair helps on every dataset, with no new labels:** 1.5–6.0 macro-F1 points over the starting model, significant on all 6. It closes 10–83% of the gap to the Random Forest.
- **Bought labels add less, on 3 of 6 datasets.** SR then AL beat self-repair alone on Landsat (+1.1), MAGIC (+2.0) and Letter (+0.7), the first and last only just (Holm-adjusted p = 0.049). On Thyroid, Page Blocks and Skin the difference was not

significant.

- **Self-repair was shown to be no worse than active learning, within 1 point, only on Letter.** On Thyroid, Landsat and Skin it was within half a point on average, too close to settle with 10 seeds. On Page Blocks and MAGIC it trailed by 2.3 and 1.8 points. The exploratory checks had found it no more than half a point behind on 5 of 6 datasets.
- **Retraining on the same labels helps little.** SR then AL beat shuffled retraining by 1.7–8.2 points on all 6 datasets. Retraining with the new rows appended in their original order changed macro-F1 by −2.6 to +0.9 points, significantly only on Letter (+0.85), and shuffled retraining by −3.4 to +1.8 points, never significantly (both post hoc).
- **Lowest class recall:** SR then AL was shown to keep it no more than 2 points below the starting model on 4 of 6 datasets, with large gains on Thyroid (+20 points) and Skin (+18). This was not shown on Page Blocks or MAGIC. Self-repair alone lowered MAGIC's lowest class recall by 2.7 points, in 9 of 10 seeds.
- **The gap to other models remains.** The Random Forest given the same labels stays 5–19 points ahead on 5 of the 6 datasets. Nearest neighbours are ahead on Landsat, MAGIC, Letter and Skin, behind on Page Blocks and far behind on Thyroid.

**Certificates and targeted change (RQ4).** In the same run we computed each accepted edit's certificates on up to 2,000 test rows (548–2,000 per dataset), and PACE-flip's explanations on 100 of them. Values are means over the 10 seeds. Retraining with the new rows appended was planned as a comparison to describe, with no test.

| Measure | Thyroid | Landsat | Page Blocks | MAGIC | Letter | Skin |
|---|---|---|---|---|---|---|
| Fix share, SR then AL (%) | 85.3 | 64.2 | 81.9 | 64.8 | 66.2 | 94.1 |
| Fix share, retrained, shuffled (%) | 51.8 | 52.4 | 42.6 | 47.2 | 50.9 | 40.5 |
| Broken, SR then AL (% of test rows) | 0.19 | 2.36 | 0.82 | 6.85 | 3.06 | 0.22 |
| Broken, retrained, shuffled (% of test rows) | 1.03 | 6.09 | 6.81 | 15.60 | 10.48 | 3.83 |
| Broken, retrained, new rows appended (% of test rows) | 0.39 | 1.77 | 3.23 | 2.97 | 2.33 | 0.49 |
| Tight certificate per edit, SR then AL (% of test rows) | 1.2 | 1.9 | 34.6 | 7.9 | 1.5 | 4.1 |
| Certified rows that changed, SR then AL (%) | 0.7 | 3.4 | 0.2 | 1.9 | 3.7 | 0.7 |

- **No prediction changed outside its certificate:** none in 46,153 accepted edits, for both the tight and the plain certificate, and none in 69,488 edits in the exploratory checks. This follows from the argument in Section 6; the count checks our code.
- **The certificates are loose.** An edit's tight certificate held 1.2–35% of the test rows, and only 0.2–3.7% of those rows changed. Over a whole repair, 5–74% of the rows fell inside some edit's certificate.

- **They do not bound changes to explanations.** Among the explained rows outside every edit's tight certificate, PACE-flip's top 3 features still changed on up to 51% (MAGIC). PACE-flip also looks at boxes that could win after a feature changes, not only at the best boxes now.
- **Repair's changes are mostly corrections.** The fix share of SR then AL beat shuffled retraining on all 6 datasets, by 12–54 points (the main test). Repair also broke fewer predictions, changed PACE-flip's top 3 on fewer explained rows (all 5 datasets tested; Skin, with 3 features, was left out as planned), and was 1.7–8.2 points more accurate.
- **The broken-prediction result depends on the retraining order.** CFMNN depends on the order of its training rows: shuffled retraining changed 2.4–30% of test predictions, and retraining with the new rows appended 0.7–8%. Against appended retraining, repair still had the higher fix share on all 6 datasets and was 2.1–7.5 points more accurate, but broke more predictions on Landsat, MAGIC and Letter.
- **Explanations still move under repair.** SR then AL changed PACE-flip's top 3 features on 26–78% of the explained rows (Skin left out).

**Deviations from the pre-registrations.** Three things differed from the plans published before the runs. First, the second explanation run (seeds 30–39) gave LIME the whole training set for its statistics, although its plan said every explainer sees only the 200 background rows; the later planned test gave LIME those rows (see Ties). Second, the conditional-replacement run was restarted after a change to the code that queries the model, shared by all methods, when 6 of its 130 jobs had finished and the results for S1 had been seen. Third, that run's planned comparison with KernelSHAP on the same rows was missing from earlier drafts; it is now reported under Faithfulness. The tests added for this version (exact values and speed; stability, ties and LIME; generic ensembles; accuracy; readable models; repair at the chosen θ) were also planned in advance, and one change was made before any run: the bagged ensemble keeps the model's own row order. Analyses marked as made after the run were not planned. The repair questions changed between runs: the first repair run tested them as first posed, and RQ3 and RQ4 as stated in Section 1 were written after it and after exploratory checks on seeds 0–2. The reframed questions were then tested in a second run planned in advance, with no change to its code or tests once it began.

**Settings.** CFMNN uses $\theta = 0.3$ (the largest box size) and $\gamma = 4$ (how fast membership fades outside a box). PACE-SHAP uses 200 background rows (71 on Wine); it is exact up to 12 features and uses 32 orders per row above that. KernelSHAP uses 200 coalition samples, and LIME 1,000 samples in regression mode, without discretisation; both explain the margin. DiCE uses its random search with 1,000 samples per search and 5 candidates, keeping the nearest valid one, and its default sparsity step. PD and EI use 50 nudges per point: noise at 5% of each feature's standard deviation, clipped to [0, 1], except that features with at most 10 distinct values are redrawn with probability 0.1. The seeds are 0–

4 for the base model, 0–9 and 30–39 for the two explanation runs, 40–49 for the later explanation checks, 10–19 for the confirmatory trust run, 20–29 for the combination run, 50–59 and 60–69 for the two planned tests added in this version, 70–89 for the readable-model test and the test of repair at the chosen θ, and, for repair, 10–19 for the first run, 0–2 for the exploratory checks and 50–59 for the second run. In the repair runs, the order ensemble has 5 CFMNNs, the Random Forest 100 trees and nearest neighbours 10 neighbours, and PACE-flip explains 100 test rows with 200 background rows. Software: Python 3.14, numpy 2.5.3, scipy 1.18.1, scikit-learn 1.9.1, pandas 3.0.5, shap 0.54.1, lime 0.2.0.1, dice-ml 0.12, interpret-core 0.7.8 (EBM) and hyperbox-brain 0.1.6 (GFMM). All runs and timings used one laptop: an Intel Core i7-8650U processor (4 cores, 8 threads, 1.9 GHz) with 16 GB of memory, under Windows 11.

## 8. Related work

**Exact and sampled Shapley values.** KernelSHAP (Lundberg and Lee) estimates Shapley values for any model, and Covert and Lee improve its estimates. For some model classes the values can be computed exactly: TreeSHAP does this for tree ensembles and also gives interaction values (Lundberg and colleagues), Zern, Broelemann and Kasneci extend it to piecewise linear trees, and Van den Broeck and colleagues and Arenas and colleagues study which models allow it. PACE-SHAP belongs to this line: it uses the fact that CFMNN scores each feature separately. Above 12 features it samples feature orders, as Štrumbelj and Kononenko proposed, using the antithetic orders that Mitchell and colleagues compare with other samplers. Interaction values in the Shapley sense go back to Grabisch and Roubens.

**Which game to explain.** A removed feature can be filled in independently of the others (the interventional game) or given them (the conditional game). Janzing, Minorics and Blöbaum, and Chen and colleagues, argue for the first when the aim is to explain the model; Aas, Jullum and Løland estimate the second. We test both.

**Faithfulness tests.** Our deletion and insertion tests follow pixel flipping (Samek and colleagues) and the curves of Petsiuk, Das and Saenko. Such tests have known pitfalls; for example, they can favour explainers that remove features the same way the test does (Hooker and colleagues; Rong and colleagues; Ju and colleagues; Blücher, Vielhaben and Strodthoff). So we report deletion and insertion, raw and normalised curves, and a conditional version. Yeh and colleagues measure a related notion of infidelity. PACE-flip is a fast form of occlusion (Zeiler and Fergus), and its global version is checked against permutation importance (Breiman; Fisher, Rudin and Dominici); Covert, Lundberg and Lee give a Shapley-based alternative.

**Boxes, prototypes and clusters.** Fuzzy min-max networks (Simpson; Gabrys and Bargiela; surveyed by Khuat, Ruta and Gabrys) are usually read through their boxes as rules; Quteishat and Lim extract such rules from a trained network. Khuat, Chen and Gabrys use an ensemble to reduce their sensitivity to the order of the training rows, the sensitivity our order ensemble measures. Explanations of related models include cluster explanations (Kauffmann and colleagues), counterfactuals for learning vector quantization (Artelt and Hammer) and for oblique trees (Carreira-Perpiñán and Hada), and explanations of max-plus networks (Enaieh, Fercoq and García). Closest to our counterfactuals, Khouna, Ferry and Vidal find exact nearest counterfactuals for tree ensembles by searching over labelled axis-aligned boxes. CFMNN differs in two ways: boxes of different classes can overlap, and membership fades gradually outside a box. So entering a box of the target class does not always change the decision, and PACE checks every answer on the model. We compare with DiCE (Mothilal, Sharma and Tan); counterfactual explanations in this sense go back to Wachter, Mittelstadt and Russell.

**Trust signals.** The model's own confidence is the usual baseline for spotting likely errors (Hendrycks and Gimpel; Geifman and El-Yaniv). Trust Score (Jiang, Kim, Guan and Gupta), disagreement between models (Lakshminarayanan, Pritzel and Blundell) and learned combiners of confidence signals (Chen, Navrátil, Iyengar and Shanmugam) are close to the standard methods and the combiner we use. Our order ensemble is a form of the disagreement studied by Jiang, Nagarajan, Baek and Kolter: between copies of a model trained on the same data in different orders.

**Repair, active learning and model updates.** Simpson's network removes overlaps between boxes of different classes with a contraction step after each expansion, and the general fuzzy min-max network of Gabrys and Bargiela keeps one; our self-repair is a gated contraction applied only where CFMNN errs. Active learning chooses which points to label (Settles); choosing where a committee of models disagrees goes back to Seung, Opper and Sompolinsky, and our order ensemble is such a committee. An update that fixes some predictions can break others: Bansal and colleagues show that this hurts people who work with the model, Srivastava and colleagues measure it across retrained models, and Yan and colleagues train new models to avoid such negative flips. Sotoudeh and Thakur repair neural networks with a proof that given points are fixed; our certificates instead bound which predictions a repair can change.

**The name.** Two other explanation methods are also called PACE: concept explanations for vision models (Wang, Tan and Wang) and a framework for plausible, actionable counterfactuals (Iakovets, Samarathunga, Horsch and Al Machot). Ours is specific to CFMNN: its explanations use the model's structure, and its counterfactuals are moves into the model's own boxes.

## 9. Next steps

- **Trust signals:** test the combined detector under real changes in the data over time, and with fewer labelled calibration rows.
- **Counterfactuals:** compare PACE and PACE-sparse with an exact search over boxes, adapted from Khouna, Ferry and Vidal's method for tree ensembles, and measure how robust each answer is to small changes.
- **Conditional replacement:** fix wider weights in advance (checks made after the run point to σ between 0.5 and 1) and test them on all 13 datasets.
- **Repair:** make self-repair a step of CFMNN's training, compare it with Simpson's contraction step, and look for certificates that are tighter and also bound changes to explanations.

## 10. Limitations

- PACE relies on CFMNN scoring each feature separately; other models would need new derivations.
- PACE-flip is not more faithful than occlusion: it gives the same result, much faster.
- PACE-SHAP is exact only up to 12 features. Beyond that it samples orders, with a 1–2% error against exact values at 32 orders per row, and a new seed changes its top 3 slightly (0.94–0.99 overlap). With 200 background rows it is usually slower than KernelSHAP, and most of its faithfulness lead over KernelSHAP comes from those rows and from keeping every feature.
- Faithfulness is measured with deletion and insertion tests on 50 points per dataset (36 for Wine) over 10 seeds per run. Both use the model's margin, the quantity PACE-SHAP and KernelSHAP explain, and replace features one at a time, which also suits occlusion-style methods such as PACE-flip. On Shuttle, Letter and Landsat, 63–70% of the inputs these tests evaluate lie off the data. Many explained points are tied, with a margin of 0 (55–79% on Letter, MAGIC and Page Blocks), and the margin ignores the nearest-centroid rule that breaks the tie; raw margin curves and untied points point the same way. With 10 seeds per comparison and Holm correction across 11–13 datasets, small differences can go undetected, so a level result does not show that two methods are equal.
- Counterfactuals were compared with one DiCE setting (its random search), and how robust each answer is to small changes was not measured.
- Conditional replacement was tested at a single setting (σ = 0.1 with 200 rows), at which the conditional game rests on very few rows.

- CFMNN is less accurate than the better of 10-nearest neighbours and a Random Forest on all 13 datasets, by 0.2 to 21.5 points, and weak on rare classes. On 12 of the 13 datasets in the trust-signal test it is also less accurate than a plain nearest-neighbour classifier. Other readable models were also more accurate on most datasets; that test ran EBM with main effects only and GFMM untuned, and several tuned models chose the end of their grid. Repair at $\theta = 0.3$ did not make up for choosing $\theta$ by validation, and it tried only 300 edits, a small share of the training errors on the large datasets (about 4,000 on MAGIC). Exact explanations of a weak model are still explanations of a weak model.
- Timings come from one laptop; we compare only speed ratios measured in the same run.
- Wine, Breast Cancer and Shuttle have too few errors to test the trust signals.
- On their own, the model's trust signals are much weaker than methods that use the training labels. The strongest detector combines both and needs labelled calibration rows.
- Repair was tested with one budget and one gate: on 6 datasets with 10 seeds for RQ3 and RQ4, and on 13 with 20 seeds in the test of repair at the chosen $\theta$. RQ3 and RQ4 were reframed after a first repair run and exploratory checks, although the reframed questions were then tested in a run planned in advance.
- Self-repair lowered MAGIC's lowest class recall by 2.7 points, and was shown to be no worse than active learning only on Letter.
- Repair's certificates are exact but loose, and do not bound changes to explanations. Repair broke fewer predictions than shuffled retraining, but more than retraining with the new rows appended on Landsat, MAGIC and Letter.

## 11. Conclusion

CFMNN scores a point feature by feature, and PACE turns that into exact, repeatable explanations. Its main explainer, PACE-SHAP, computes the Shapley values that KernelSHAP estimates, exactly for up to 12 features; its values match those of a general exact method, 16–30× faster. It was more stable than KernelSHAP and never significantly less faithful; its lead over KernelSHAP's usual setting, on 6 of 11 datasets, comes mostly from using more background rows and every feature, and given the same rows it was level on most datasets and 3.5–10× faster. PACE-flip gives brute-force occlusion's answer (up to floating-point rounding) in 1.4 ms or less, 25–134× faster even than occlusion run in one batch, but is less faithful than the Shapley methods on most datasets. To flag the model's likely mistakes, the model's own signals beat its plain margin but not standard methods that use the training labels; adding the disagreement of CFMNNs trained in other orders to those methods helps on most datasets, but CFMNNs trained on bootstrap samples

help as much. The explanation-based signal adds nothing, so we keep it only as a diagnostic. PACE's counterfactuals are confirmed by the model, land closer to real data than DiCE's, and are far faster, while DiCE's change fewer features. A sparse variant, PACE-sparse, changes fewer features than DiCE on 10 of 11 datasets, but lands farther from real data than PACE. Repairing the model needs few or no new labels: shrinking the wrong box at the model's own training errors raised macro-F1 on all 6 datasets tested, winning back 10–83% of the gap to a Random Forest, and bought labels added less, on 3. For each box edit we can find the inputs whose prediction it can change, and repair's changes were corrections far more often than retraining's, though that set is loose and does not cover explanations. All of this explains a transparent model that, on every dataset in our base run, is 0.2–21.5 points less accurate than the better of nearest neighbours and a Random Forest; repair narrows that gap but does not close it.

## References


- P. K. Simpson, "Fuzzy min-max neural networks — Part 1: Classification," *IEEE Transactions on Neural Networks*, 3(5):776–786, 1992. doi:10.1109/72.159066
- V. Aadarsh and P. S. V. S. Sai Prasad, "Scalable Centroid Based Fuzzy Min-Max Neural Network Ensemble Classifier Using MapReduce," *Pattern Recognition and Machine Intelligence (PReMI 2023)*, Springer, 2023. doi:10.1007/978-3-031-45170-6_62
- M. T. Ribeiro, S. Singh and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," *KDD*, 2016 (LIME).
- S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017 (SHAP).
- R. K. Mothilal, A. Sharma and C. Tan, "Explaining Machine Learning Classifiers through Diverse Counterfactual Explanations," *FAT**, 2020 (DiCE).
- H. Jiang, B. Kim, M. Y. Guan and M. Gupta, "To Trust or Not to Trust a Classifier," *NeurIPS*, 2018 (Trust Score).
- D. Alvarez-Melis and T. S. Jaakkola, "On the Robustness of Interpretability Methods," ICML Workshop on Human Interpretability in Machine Learning, 2018 (arXiv:1806.08049).
- H. Chen, J. D. Janizek, S. Lundberg and S.-I. Lee, "True to the Model or True to the Data?," arXiv:2006.16234, 2020.
- K. Aas, M. Jullum and A. Løland, "Explaining individual predictions when features are dependent: More accurate approximations to Shapley values," *Artificial Intelligence*, 2021.
- D. Chicco and G. Jurman, "Survival prediction of patients with sepsis from age, sex, and septic episode number alone," *Scientific Reports*, 2020.

- S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, 2020.
- I. Covert and S.-I. Lee, "Improving KernelSHAP: Practical Shapley value estimation using linear regression," AISTATS, 2021.
- A. Zern, K. Broelemann and G. Kasneci, "Interventional SHAP values and interaction values for piecewise linear regression trees," AAAI, 2023.
- G. Van den Broeck, A. Lykov, M. Schleich and D. Suciu, "On the tractability of SHAP explanations," AAAI, 2021.
- M. Arenas, P. Barceló, L. Bertossi and M. Monet, "On the complexity of SHAP-score-based explanations: Tractability via knowledge compilation and non-approximability results," *Journal of Machine Learning Research*, 2023.
- E. Štrumbelj and I. Kononenko, "Explaining prediction models and individual predictions with feature contributions," *Knowledge and Information Systems*, 2014.
- R. Mitchell, J. Cooper, E. Frank and G. Holmes, "Sampling permutations for Shapley value estimation," *Journal of Machine Learning Research*, 2022.
- M. Grabisch and M. Roubens, "An axiomatic approach to the concept of interaction among players in cooperative games," *International Journal of Game Theory*, 1999.
- D. Janzing, L. Minorics and P. Blöbaum, "Feature relevance quantification in explainable AI: A causal problem," AISTATS, 2020.
- W. Samek, A. Binder, G. Montavon, S. Lapuschkin and K.-R. Müller, "Evaluating the visualization of what a deep neural network has learned," *IEEE Transactions on Neural Networks and Learning Systems*, 2017.
- V. Petsiuk, A. Das and K. Saenko, "RISE: Randomized input sampling for explanation of black-box models," BMVC, 2018.
- S. Hooker, D. Erhan, P.-J. Kindermans and B. Kim, "A benchmark for interpretability methods in deep neural networks," NeurIPS, 2019.
- Y. Rong, T. Leemann, V. Borisov, G. Kasneci and E. Kasneci, "A consistent and efficient evaluation strategy for attribution methods," ICML, 2022.
- Y. Ju, Y. Zhang, Z. Yang, Z. Jiang, K. Liu and J. Zhao, "Logic traps in evaluating attribution scores," ACL, 2022.
- S. Blücher, J. Vielhaben and N. Strodthoff, "Decoupling pixel flipping and occlusion strategy for consistent XAI benchmarks," *Transactions on Machine Learning Research*, 2024.
- C.-K. Yeh, C.-Y. Hsieh, A. S. Suggala, D. I. Inouye and P. Ravikumar, "On the (in)fidelity and sensitivity of explanations," NeurIPS, 2019.
- M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," ECCV, 2014.

- L. Breiman, "Random forests," *Machine Learning*, 2001.
- L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees*, Wadsworth, 1984.
- Y. Lou, R. Caruana and J. Gehrke, "Intelligible models for classification and regression," KDD, 2012.
- H. Nori, S. Jenkins, P. Koch and R. Caruana, "InterpretML: A unified framework for machine learning interpretability," arXiv:1909.09223, 2019.
- A. Fisher, C. Rudin and F. Dominici, "All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously," *Journal of Machine Learning Research*, 2019.
- I. Covert, S. M. Lundberg and S.-I. Lee, "Understanding global feature contributions with additive importance measures," NeurIPS, 2020.
- B. Gabrys and A. Bargiela, "General fuzzy min-max neural network for clustering and classification," *IEEE Transactions on Neural Networks*, 2000.
- T. T. Khuat, D. Ruta and B. Gabrys, "Hyperbox-based machine learning algorithms: A comprehensive survey," *Soft Computing*, 2021.
- T. T. Khuat, F. Chen and B. Gabrys, "An improved online learning algorithm for general fuzzy min-max neural network," IJCNN, 2020.
- A. Quteishat and C. P. Lim, "A modified fuzzy min-max neural network with rule extraction and its application to fault detection and classification," *Applied Soft Computing*, 8(2):985–995, 2008.
- J. Kauffmann, M. Esders, L. Ruff, G. Montavon, W. Samek and K.-R. Müller, "From clustering to cluster explanations via neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, 2024.
- A. Artelt and B. Hammer, "Efficient computation of counterfactual explanations of LVQ models," ESANN, 2020.
- Enaieh, Fercoq and García, "On the explainability of max-plus neural networks," arXiv:2605.00889, 2026.
- M. Á. Carreira-Perpiñán and S. S. Hada, "Counterfactual explanations for oblique decision trees: Exact, efficient algorithms," AAAI, 2021.
- Khouna, Ferry and Vidal, "Counterfactual maps: What they are and how to find them," arXiv:2602.09128, 2026.
- S. Wachter, B. Mittelstadt and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard Journal of Law & Technology*, 2018.
- D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," ICLR, 2017.

- Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," NeurIPS, 2017.
- B. Lakshminarayanan, A. Pritzel and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," NeurIPS, 2017.
- Y. Jiang, V. Nagarajan, C. Baek and J. Z. Kolter, "Assessing generalization of SGD via disagreement," ICLR, 2022.
- T. Chen, J. Navrátil, V. Iyengar and K. Shanmugam, "Confidence scoring using whitebox meta-models with linear classifier probes," AISTATS, 2019.
- H. Wang, S. Tan and H. Wang, "Probabilistic conceptual explainers: Trustworthy conceptual explanations for vision foundation models," ICML, 2024.
- Iakovets, Samarathunga, Horsch and Al Machot, "PACE: A neuro-symbolic framework for plausible and actionable counterfactual explanations," arXiv:2607.01306, 2026.
- B. Settles, "Active learning literature survey," Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009.
- H. S. Seung, M. Opper and H. Sompolinsky, "Query by committee," COLT, 1992.
- G. Bansal, B. Nushi, E. Kamar, D. S. Weld, W. S. Lasecki and E. Horvitz, "Updates in human-AI teams: Understanding and addressing the performance/compatibility tradeoff," AAAI, 2019.
- M. Srivastava, B. Nushi, E. Kamar, S. Shah and E. Horvitz, "An empirical analysis of backward compatibility in machine learning systems," KDD, 2020.
- S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia and S. Soatto, "Positive-congruent training: Towards regression-free model updates," CVPR, 2021.
- M. Sotoudeh and A. V. Thakur, "Provable repair of deep neural networks," PLDI, 2021.
- M. Kelly, R. Longjohn and K. Nottingham, "The UCI Machine Learning Repository," https://archive.ics.uci.edu.